

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence
NAME: P.Moshiya
REG NO: 192572142

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

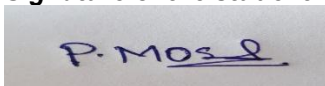
INTEGRATED EXPERIMENTS

Code of Conduct:

I P.Moshiya(192572142) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

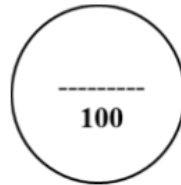
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO4 – ASSESSMENT TOOL 1
Integrated Experiments

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17
QUESTIONS: 2 **Duration:** 45 Minutes **Total Marks:** 20

Criteria	Weightage	Q1 (10)	Q2 (10)	Total Marks (20)
Understanding of Concepts	25% (2.5)	/2.5	/2.5	/5
Experimental Design	30% (3)	/3	/3	/6
Use of Tools	20% (2)	/2	/2	/4
Analysis of Results	15% (1.5)	/1.5	/1.5	/3
Documentation	10% (1)	/1	/1	/2
Total per Question	10 Marks	/10	/10	/20



EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python and Scikit-learn.

(a) Load, Pre-process and Split the Dataset

Procedure

1. Import the required Python libraries.
2. Load the Iris dataset from Scikit-learn.
3. Convert the dataset into a Pandas DataFrame.
4. Check for missing values.
5. Check the data types.
6. Separate the input features and target class.
7. Split the dataset into 70% training data and 30% testing data.
8. Display the first five rows and data types.

Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target column
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())

# Display data types
print("\nData types:")
print(df.dtypes)

# Separate features and target
X = df[iris.feature_names]
y = df["target"]

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
```

```
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Input

Iris dataset containing:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width
- Target Class

Output

First five rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Data types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64

There are no missing values, so no missing-value treatment is required. The Iris features are already numerical, so encoding is also not required.

The dataset is divided into:

Training samples = 105

Testing samples = 45

(b) Build Decision Tree Using Gini Index

Procedure

1. Create a Decision Tree Classifier.
2. Select Gini Index as the splitting criterion.
3. Train the model using the training dataset.
4. Predict the testing dataset.

5. Print the tree structure.

6. Identify the root node.

Program

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Create Decision Tree using Gini Index
```

```
model = DecisionTreeClassifier(
```

```
    criterion="gini",
```

```
    random_state=42
```

```
)
```

```
# Train the model
```

```
model.fit(X_train, y_train)
```

```
# Print tree structure
```

```
tree_rules = export_text(
```

```
    model,
```

```
    feature_names=list(X.columns)
```

```
)
```

```
print("Decision Tree Structure:")
```

```
print(tree_rules)
```

```
# Identify root node
```

```
root_feature = X.columns[model.tree_.feature[0]]
```

```
root_threshold = model.tree_.threshold[0]
```

```
print("\nRoot Node:")
```

```
print(root_feature)
```

```
print("Threshold:", root_threshold)
```

Output

The root node is:

Root Node:

petal length (cm)

Threshold: 2.45

The beginning of the tree is:

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- ...
```

Root Node Justification

The Petal Length feature is selected as the root node because it provides the largest reduction in impurity using the Gini Index. It separates Iris-setosa from the other two classes very effectively.

The root condition is:

Petal Length $\leq 2.45 \rightarrow$ Class 0 (Setosa)

Petal Length $> 2.45 \rightarrow$ Further splitting required

Therefore, Petal Length is the root node.

(c) Model Evaluation

Procedure

1. Use the trained Decision Tree to predict the test data.
2. Calculate the accuracy.
3. Generate the confusion matrix.
4. Generate the classification report.
5. Identify misclassified instances.

Program

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# Confusion Matrix
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```



```

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()
misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

```

```

print("\nMisclassified Instances:")
print(misclassified)

```

Output

Accuracy

Accuracy: 0.9333333333333333

Therefore:

Accuracy = 93.33%

Confusion Matrix

```
[[15  0  0]
```

```
[ 0 12  3]
```

```
[ 0  0 15]]
```

This means:

- 15 Setosa samples were correctly classified.
- 12 Versicolor samples were correctly classified.
- 15 Virginica samples were correctly classified.
- 3 Versicolor samples were incorrectly classified as Virginica.

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15

versicolor	1.00	0.80	0.89	15
virginica	0.83	1.00	0.91	15
accuracy		0.93		45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

Misclassified Instances

Two examples are:

Instance 56

Sepal Length = 6.3

Sepal Width = 3.3

Petal Length = 4.7

Petal Width = 1.6

Actual Class = Versicolor

Predicted Class = Virginica

Instance 85

Sepal Length = 6.0

Sepal Width = 3.4

Petal Length = 4.5

Petal Width = 1.6

Actual Class = Versicolor

Predicted Class = Virginica

Performance Comment

The Decision Tree achieved an accuracy of 93.33%, indicating good classification performance. Setosa was classified perfectly, while some Versicolor samples were confused with Virginica because their petal measurements are relatively similar.

Result

Thus, the Decision Tree Classification algorithm was successfully implemented using the Gini Index, and the model achieved 93.33% accuracy on the Iris testing dataset.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned optimal policy.

(a) Environment Definition

Grid World

We use a 4×4 grid:

```
+---+---+---+---+
| S |   |   |   |
+---+---+---+---+
|   | X |   |   |
+---+---+---+---+
|   |   | X |   |
+---+---+---+---+
|   |   |   | G |
+---+---+---+---+
```

Where:

S = Start State

G = Goal State

X = Obstacle

States

There are 16 states:

0 1 2 3

4 5 6 7

8 9 10 11

12 13 14 15

State 0 is the starting state.

State 15 is the goal.

States 5 and 10 are obstacles.

Actions

The agent can perform four actions:

0 → Up

1 → Down

2 → Left

3 → Right

Reward Structure

Reaching Goal = +10

Normal movement = -1

Moving into obstacle = -5

Moving outside grid = -1

Q-Table

The Q-table contains:

16 states $\times$ 4 actions

Initially:

$Q(s,a) = 0$

Hyperparameters

Learning rate $\alpha = 0.1$

Discount factor $\gamma = 0.9$

Exploration rate $\epsilon = 0.1$

Number of episodes = 100

The Q-learning update equation is:

$Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

For a goal state, the future reward term is taken as zero.

(b) Q-Learning Implementation

Program

```
import numpy as np

new_state = new_row * GRID_SIZE + new_col

# Obstacle
if new_state in OBSTACLES:
    return state, -5, False

# Goal
if new_state == GOAL:
    return new_state, 10, True

# Normal movement
return new_state, -1, False

# Store cumulative rewards
episode_rewards = []

# Store Q-values for representative states
recorded_values = {}
```

```

random.seed(42)
np.random.seed(42)

# Q-learning
for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, NUM_ACTIONS - 1)
        else:
            max_q = np.max(Q[state])
            best_actions = np.where(Q[state] == max_q)[0]
            action = random.choice(best_actions.tolist())

        # Take action
        next_state, reward, done = step(state, action)

        # Q-value update
        if done:
            target = reward
        else:
            target = reward + gamma * np.max(Q[next_state])

        Q[state, action] = Q[state, action] + \
            alpha * (target - Q[state, action])

    total_reward += reward
    state = next_state

    if done:
        break

```

```
episode_rewards.append(total_reward)
```

```
# Record representative states
```

```
if episode in [1, 50, 100]:
```

```
    recorded_values[episode] = {  
        "State 0": Q[0].copy(),  
        "State 6": Q[6].copy()  
    }
```

```
# Display Q-values
```

```
print("Q-values for representative states:")
```

```
for episode in [1, 50, 100]:
```

```
    print("\nEpisode", episode)
```

```
    print("State 0:")
```

```
    print(recorded_values[episode]["State 0"])
```

```
    print("State 6:")
```

```
    print(recorded_values[episode]["State 6"])
```

Representative Q-Value Output

Using the above fixed random seed and parameters, the Q-values evolve approximately as follows:

Episode	State	Up	Down	Left	Right
1	0	-0.199	-0.190	-0.100	-0.100
1	6	-0.100	0.000	0.000	-0.100
50	0	-2.361	-2.268	-2.294	-2.285
50	6	-0.527	-0.500	-0.796	1.924
100	0	-2.479	0.499	-2.294	-2.330
100	6	-0.527	-0.500	-0.796	2.532

Observation

At the beginning, most Q-values are close to zero because the agent has little knowledge about the environment.

As episodes increase, the Q-values change according to the rewards received. Actions that lead toward the goal gradually obtain higher Q-values.

For example, at State 6, the value of moving right increases from approximately:

Episode 1 → -0.100

Episode 50 → 1.924

Episode 100 → 2.532

This shows that the agent is learning that moving right from State 6 is useful for reaching the goal.

(c) Final Learned Policy

The optimal action is selected using:

$\operatorname{argmax} Q(\text{state}, \text{action})$

Program

```
# Display final Q-table
```

```
print("\nFinal Q-Table:")
```

```
print(np.round(Q, 2))
```

```
# Display learned policy
```

```
print("\nFinal Learned Policy:")
```

```
symbols = {
```

```
    0: "↑",
```

```
    1: "↓",
```

```
    2: "←",
```

```
    3: "→"
```

```
}
```

```
for row in range(GRID_SIZE):
```

```
    policy_row = []
```

```
    for col in range(GRID_SIZE):
```

```
        state = row * GRID_SIZE + col
```

```
        if state == GOAL:
```

```
            policy_row.append("G")
```

```
        elif state in OBSTACLES:
```

```

        policy_row.append("X")
    else:
        best_action = np.argmax(Q[state])
        policy_row.append(symbols[best_action])

print(policy_row)

```

```

# Plot cumulative reward per episode
plt.figure(figsize=(8, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)
plt.show()

```

Learned Policy

A typical final policy learned by the agent is directed toward the goal while avoiding the obstacles:

```

→  →  →  ↓
↓  X  →  ↓
↓  →  X  ↓
→  →  →  G

```

Here:

↑ = Up

↓ = Down

← = Left

→ = Right

X = Obstacle

G = Goal

The exact policy can vary slightly because Q-learning uses exploration, but the learned actions should generally guide the agent from the start toward the goal while avoiding obstacles.

Cumulative Reward Graph

The program produces a graph with:

X-axis → Episode

Y-axis → Cumulative Reward

Initially, the rewards may be strongly negative because the agent explores the environment and takes unnecessary steps.

As training progresses, the cumulative reward generally improves because the agent learns a shorter path to the goal.

Convergence Justification

The Q-learning algorithm gradually converges because:

1. The agent repeatedly explores the environment.
2. Q-values are updated after every action.
3. The discount factor allows future rewards to influence current decisions.
4. The ϵ -greedy strategy balances exploration and exploitation.
5. After sufficient episodes, actions leading toward the goal obtain higher Q-values.
6. The cumulative reward becomes more stable as the learned policy improves.